

PORTFOLIO — FULL PROJECT DOSSIER

트래픽 대응 티켓 예매 서비스 포트폴리오 통합 문서

프로젝트 개요부터 인프라 설계, 위협모델링·공격 대응, 계정 보안, 부하테스트 계획, 최종 산출물까지 지금까지 논의한 전체 내용을 하나로 정리한 문서다.

작성일 2026.07 · 대상 한강 이글스 티켓 예매(데모) · 환경 VMware 신규 VM(홈랩과 분리)

Part 1. 프로젝트 개요	1.1 개요 및 필요성 · 1.2 진행 로드맵
Part 2. 인프라 설계	2.1 서버 구성도 · 2.2 서버별 설치 · 2.3 배포 · 2.4 방화벽 · 2.5 보안효과
Part 3. 위협 대응 및 계정 보안	3.1 공격 대응 로직 · 3.2 인증 보안 설계
Part 4. 검증 계획	4.1 Phase별 실행계획 · 4.2 성공 판단 기준
Part 5. 자체 모의해킹 검증	5.1 목적/범위 · 5.2 방법론 · 5.3 도구 · 5.4 실행방법 · 5.5 판단기준 · 5.6 보고서템플릿 · 5.7 윤리
Part 6. 산출물	6.1 문서 · 6.2 구현 · 6.3 시각/발표자료
Part 7. 캡처 자료 체크리스트	7.1~7.7 단계별 캡처 항목

Part 1 — 프로젝트 개요

1.1 개요 및 필요성 (보안 관점)

일반적인 웹 서비스 포트폴리오에는 "기능이 동작하는가"에 초점을 맞추지만, 보안 관점에서 중요한 질문은 하나 더 있다. "이 서비스가 공격받거나 트래픽이 폭증해도 안전하게 버티는가." 티켓 예매 서비스는 이 질문이 실제로 서비스 존폐를 가르는 대표적 도메인이다. 순간적인 트래픽 폭증은 그 자체로 가용성(Availability)을 위협하는 DoS성 상황이고, 좌석 중복판매는 무결성(Integrity) 문제이며, 회원·결제 정보 처리 과정은 기밀성(Confidentiality) 문제다. 즉 이 프로젝트 하나에 정보보안의 3요소(CIA)가 모두 걸려 있다.

CIA 요소	이 프로젝트에서의 위협	대응 설계
기밀성 (Confidentiality)	회원정보·결제정보 유출, 내부망 스니핑	TLS, 네트워크 계층 분리, DB 최소권한 계정
무결성(Integrity)	좌석 동시 클릭으로 인한 중복판매(Race Condition)	Redis 원자연산(SETNX/DECR)으로 구조적 방지
가용성(Availability)	예매 오픈 순간 트래픽 폭증, 매크로/봇에 의한 DoS	대기열(Rate limiting), 로드밸런싱, 이상탐지

설계 원칙 — 이 문서 전반에 걸쳐 "외부에 노출되는 통로는 최소한으로, 내부 접근은 계정·네트워크 단위로 세분화"하는 최소권한 원칙(PoLP)과 방어 심층화(Defense in Depth)를 일관되게 적용한다.

1.2 진행 로드맵

단계	내용	핵심 작업	산출물
Phase 1	인프라 최소 구축	Bastion / WAS / Redis / DB VM 구축, 네트워크 분리, 방화벽 설정	VM 구성표, ufw 정책 캡처
Phase 2	예약 API 구현	좌석 예약 API + Redis 원자연산(DECR/SETNX) 로직 구현	API 코드, Redis 로직 설명
Phase 3	시나리오 3 — 동시성 검증	좌석 1석에 100건 동시 요청 → K6로 재현, DB 결과 확인	동시성 검증 리포트
Phase 4	인프라 확장	queue-gate, web-lb, was-02 추가 구축	확장된 아키텍처 다이어그램
Phase 5	시나리오 2 — 폭증 트래픽 비교	대기열 ON/OFF 상태로 동일 시나리오 K6 실행, 결과 비교	부하테스트 결과 보고서(그래프 포함)
Phase 6	문서화 및 정리		포트폴리오 문서 세트, README

Part 2 — 인프라 설계

2.1 서버 구성도



별도 접근 경로	내용
bastion (10.10.0.10)	SSH 유일 진입점. 관리자는 bastion을 거쳐서만 위 모든 서버에 22번 포트로 접근. 다른 경로의 SSH는 전부 차단.

인프라 목록

서버	IP	역할	비고
bastion	10.10.0.10	SSH 진입점	NIC 2개(외부용/내부용), Phase 1부터 필요
queue-gate	10.10.0.15	대기열 · Rate limiting	Phase 4에서 추가
web-lb	10.10.0.20	로드밸런서 · TLS 종료	Phase 4에서 추가
was-01 / was-02	10.10.0.11 / .12	예약 API 서버	was-01은 Phase 1부터, was-02는 Phase 4
redis	10.10.0.30	좌석 재고 원자연산 · 대기열	Phase 1부터 필요
	10.10.0.40 / .41	예약 데이터 저장	Replica는 Phase 4에서 추가

db-01 (Primary/Replica)			
monitor	10.10.0.50	ELK · Suricata	선택, 여유 있을 때 추가

2.2 서버별 설치 항목 및 명령어

2.2.1 bastion (10.10.0.10)

설치 항목: fail2ban, ufw, SSH 하드닝

```
sudo apt update && sudo apt install -y fail2ban ufw

# SSH는 키 인증만 허용 (비밀번호 로그인 차단)
sudo sed -i 's/#PermitRootLogin.* /PermitRootLogin no/' /etc/ssh/sshd_config
sudo sed -i 's/#PasswordAuthentication.* /PasswordAuthentication no/' /etc/ssh/sshd_config
sudo systemctl restart sshd

sudo ufw default deny incoming
sudo ufw allow from <관리자_PC_IP> to any port 22 proto tcp
sudo ufw enable

sudo systemctl enable --now fail2ban
```

설정	효과
키 인증만 허용	비밀번호 무차별 대입(Brute-force) 공격 자체가 통하지 않게 됨
ufw로 관리자 IP만 22 허용	진입점을 물리적으로 하나로 좁혀 공격 표면(Attack Surface) 최소화
fail2ban	반복 로그인 실패 IP를 자동 차단해 무차별 대입 공격을 조기에 무력화

2.2.2 queue-gate (10.10.0.15)

설치 항목: Nginx, Rate limiting 설정

```
sudo apt install -y nginx
sudo ufw default deny incoming
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw allow from 10.10.0.10 to any port 22 proto tcp
sudo ufw enable
```

```
# /etc/nginx/conf.d/queue.conf
limit_req_zone $binary_remote_addr zone=global:20m rate=10r/s;
limit_conn_zone $binary_remote_addr zone=addr:10m;

server {
    listen 80;
    limit_req zone=global burst=20 nodelay;
    limit_conn addr 5;
    location / { proxy_pass http://10.10.0.20; }
}
```

설정	효과
limit_req (초당 10건 제한)	한 IP가 순간적으로 대량 요청을 보내도 뒷단(web-lb, WAS)까지 도달하는 양을 제한 → 과부하로 인한 다운 방지
limit_conn (동시연결 5개)	단일 IP의 동시 커넥션 점유를 제한해 다른 사용자의 접근 기회 보장
80/443만 외부 개방	그 외 모든 내부 포트는 인터넷에서 절대 접근 불가

2.2.3 web-lb (10.10.0.20)

설치 항목: Nginx, 로드밸런싱, TLS

```
sudo apt install -y nginx
sudo ufw default deny incoming
sudo ufw allow from 10.10.0.15 to any port 80 proto tcp
sudo ufw allow from 10.10.0.15 to any port 443 proto tcp
sudo ufw allow from 10.10.0.10 to any port 22 proto tcp
sudo ufw enable
```

```
# /etc/nginx/conf.d/upstream.conf
upstream was_pool {
    least_conn;
    server 10.10.0.11:3000 max_fails=3 fail_timeout=10s;
    server 10.10.0.12:3000 max_fails=3 fail_timeout=10s;
}
server {
    listen 443 ssl;
    ssl_certificate /etc/nginx/ssl/cloudflare-origin.pem;
    ssl_certificate_key /etc/nginx/ssl/cloudflare-origin.key;

    root /var/www/ticket; # 정적 페이지(홈페이지) 서빙
    index index.html;

    location / { try_files $uri $uri/ /index.html; }
    location /api/ { proxy_pass http://was_pool; }
}
```

설정	효과
출발지를 queue-gate로 제한	대기열을 거치지 않은 트래픽은 web-lb에 도달 자체가 불가능 → 우회 차단
least_conn 로드밸런싱	WAS 두 대에 부하를 균등 분산해 한 대에 몰려 다운되는 상황 방지
max_fails/fail_timeout	응답 없는 WAS를 자동으로 풀에서 제외해 장애 전파 방지
TLS 종료(Origin 인증서)	구간 암호화로 스니핑에 의한 정보 노출 방지

2.2.4 was-01 / was-02 (10.10.0.11 / .12)

설치 항목: Node.js, 예약 API 애플리케이션

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs

sudo ufw default deny incoming
sudo ufw allow from 10.10.0.20 to any port 3000 proto tcp
sudo ufw allow from 10.10.0.10 to any port 22 proto tcp
sudo ufw enable

sudo systemctl enable --now ticket-was # 앱을 systemd 서비스로 상시 구동
```

설정	효과
web-lb IP에서만 3000 허용	인터넷은 물론 다른 내부 서버에서도 WAS에 직접 접근 불가 → 앱 서버가 항상 LB를 거치도록 강제
systemd 상시 구동	프로세스 비정상 종료 시 자동 재시작되어 가용성 확보
서버측 금액/좌석 재계산	클라이언트가 보낸 값을 신뢰하지 않아 위반조건 요청으로 인한 결제 조작 방지

2.2.5 redis (10.10.0.30)

설치 항목: Redis, 인증 설정

```
sudo apt install -y redis-server
sudo sed -i "s/bind 127.0.0.1*/bind 10.10.0.30/" /etc/redis/redis.conf
sudo sed -i "s/# requirepass */requirepass <강력한비밀번호>/" /etc/redis/redis.conf
sudo systemctl restart redis-server

sudo ufw default deny incoming
sudo ufw allow from 10.10.0.11 to any port 6379 proto tcp
```

```
sudo ufw allow from 10.10.0.12 to any port 6379 proto tcp
sudo ufw allow from 10.10.0.10 to any port 22 proto tcp
sudo ufw enable
```

설정	효과
requirepass	내부망 침투가 발생해도 비밀번호 없이는 캐시 데이터 접근·조작 불가
bind를 내부 IP로 고정	외부는 물론 다른 대역에서도 Redis 포트 자체에 연결 불가
SETNX/DECR 원자연산 사용	여러 요청이 동시에 들어와도 좌석 하나가 한 번만 처리되는 것을 구조적으로 보장 (동시성 취약점 원천 차단)

2.2.6 db-01 Primary (10.10.0.40) / db-01-replica (10.10.0.41)

설치 항목: MariaDB, 계정 최소권한, 복제(Replication)

```
# Primary
sudo apt install -y mariadb-server
sudo mysql_secure_installation
sudo sed -i "s/bind-address.*/bind-address = 10.10.0.40/" /etc/mysql/mariadb.conf.d/50-server.cnf
sudo systemctl restart mariadb

sudo ufw default deny incoming
sudo ufw allow from 10.10.0.11 to any port 3306 proto tcp
sudo ufw allow from 10.10.0.12 to any port 3306 proto tcp
sudo ufw allow from 10.10.0.41 to any port 3306 proto tcp # 복제 트래픽
sudo ufw allow from 10.10.0.10 to any port 22 proto tcp
sudo ufw enable
```

```
CREATE USER 'was_writer'@'10.10.0.%' IDENTIFIED BY '비밀번호';
GRANT SELECT, INSERT, UPDATE ON ticket.* TO 'was_writer'@'10.10.0.%';
```

```
CREATE USER 'replicator'@'10.10.0.41' IDENTIFIED BY '비밀번호';
GRANT REPLICATION SLAVE ON *.* TO 'replicator'@'10.10.0.41';
```

```
# Replica
sudo apt install -y mariadb-server
sudo sed -i "s/bind-address.*/bind-address = 10.10.0.41/" /etc/mysql/mariadb.conf.d/50-server.cnf
sudo systemctl restart mariadb
# CHANGE MASTER TO ... 로 Primary와 연결

sudo ufw default deny incoming
sudo ufw allow from 10.10.0.11 to any port 3306 proto tcp
sudo ufw allow from 10.10.0.12 to any port 3306 proto tcp
sudo ufw allow from 10.10.0.10 to any port 22 proto tcp
sudo ufw enable
```

```
CREATE USER 'was_reader'@'10.10.0.%' IDENTIFIED BY '비밀번호';
GRANT SELECT ON ticket.* TO 'was_reader'@'10.10.0.%'; # 읽기 전용
```

설정	효과
계정별 권한 분리(writer/reader/replicator)	한 계정 정보가 유출돼도 그 계정 권한 범위 밖으로는 피해가 번지지 않음(PoLP)
bind-address 내부 IP 고정	DB가 외부·타 대역에 원천적으로 노출되지 않음
Primary/Replica 분리	읽기 부하를 Replica로 분산해 좌석 조회 폭주 시에도 예약(쓰기) 처리 지연 방지

2.2.7 monitor (10.10.0.50)

설치 항목: Suricata, ELK(Elasticsearch/Logstash/Kibana)

```
sudo apt install -y suricata
# ELK는 Elastic 공식 저장소 등록 후 설치
# elasticsearch, logstash, kibana 순으로 설치
```

```
sudo ufw default deny incoming
sudo ufw allow from 10.10.0.0/24 to any port 5044 proto tcp
```

```
sudo ufw allow from 10.10.0.10 to any port 5601 proto tcp # Kibana는 bastion 경유만
sudo ufw allow from 10.10.0.10 to any port 22 proto tcp
sudo ufw enable
```

설정	효과
Kibana를 bastion IP에서만 허용	관리용 대시보드가 인터넷은 물론 내부 다른 서버에서도 접근 불가 → 모니터링 시스템 자체가 공격 표적이 되는 것 방지
Suricata 이상탐지	비정상 반복 요청(봇/매크로) 패턴을 실시간 탐지해 조기 대응 가능
ELK 로그 상관분석	여러 서버의 로그를 한 곳에 모아 공격 여부를 종합적으로 판단(단일 로그만으로는 놓치는 패턴 포착)

2.3 웹페이지(정적 사이트) 배포 방법

제작한 예매 페이지(HTML 단일 파일)는 별도 서버 프로그램 없이 **Nginx가 정적 파일로 직접 서빙**하고, 좌석 예약 같은 동적 기능만 `/api/` 경로로 WAS에 위임하는 구조다.

배포 절차

```
# 1) 로컬에서 web-lb 서버로 파일 전송
scp baseball-ticket.html 관리자계정@bastion_IP:/tmp/
# bastion을 거쳐 web-lb로 재전송 (직접 SSH 불가하므로 ProxyJump 사용)
scp -o ProxyJump=관리자계정@bastion_IP baseball-ticket.html 관리자계정@10.10.0.20:/tmp/

# 2) web-lb에서 웹 루트로 이동
sudo mkdir -p /var/www/ticket
sudo mv /tmp/baseball-ticket.html /var/www/ticket/index.html
sudo chown -R www-data:www-data /var/www/ticket
sudo chmod -R 755 /var/www/ticket

# 3) Nginx 설정 검증 및 반영 (3-3의 upstream.conf 적용 상태)
sudo nginx -t
sudo systemctl reload nginx

# 4) 접속 확인 (내부망에서)
curl -I http://10.10.0.20
```

API 연동 시 참고 — 지금 만든 페이지의 예매 로직(좌석선택, 결제)은 프론트엔드 안에서만 동작하는 데모 상태다. 실제로 서버와 연동하려면 `fetch('/api/book-seat', { ... })` 형태로 WAS의 예약 API를 호출하도록 JS를 수정해야 하며, 이 요청은 Nginx의 `location /api/` 블록을 통해 `was_pool`로 자동 프록시된다.

배포 방식	이유
정적 파일은 Nginx가 직접 서빙	WAS(애플리케이션 서버)까지 갈 필요 없어 빠르고, WAS 부하를 API 요청에만 집중시킬 수 있음
ProxyJump으로 bastion 경유 전송	web-lb에 직접 SSH를 열지 않고도 파일 전송 가능 → 관리 목적이라도 예외 없이 진입점을 하나로 유지

2.4 방화벽 및 포트 설정 정리

서버	열린 포트	허용 출발지	용도
bastion	22	관리자 PC(고정 IP)	SSH 진입점
queue-gate	80, 443	인터넷(터널 경유) / 전체	대기열 처리
queue-gate	22	bastion	관리용
web-lb	80, 443	queue-gate	LB 수신
web-lb	22	bastion	관리용

was-01/02	3000	web-lb	API 수신
was-01/02	22	bastion	관리용
redis	6379	was-01, was-02	재고/대기열 캐시
db-01 Primary	3306	was-01, was-02, db-replica	쓰기 + 복제
db-01-replica	3306	was-01, was-02	읽기 전용 계정만
monitor	5044	전체 내부 VM	로그 수집
monitor	5601	bastion만	Kibana

공통 원칙 — 모든 서버는 `ufw default deny incoming`을 기본으로 하고, 위 표에 명시된 출발지·포트만 화이트리스트로 허용한다. 이렇게 하면 "허용되지 않은 것은 전부 차단"이 기본 상태가 되어, 설정을 빠뜨려도 안전한 방향(단침)으로 실패한다.

2.5 보안 설정별 효과 요약

설정	무엇을 하는가	얻는 효과
Bastion 단일 진입점	SSH 접근을 한 서버로만 집중	공격 표면 최소화, 관리 접근 감사(audit) 용이
ufw 화이트리스트(전 서버)	필요한 출발지·포트만 허용	내부 서버 간 이동(lateral movement) 차단, 피해 확산 방지
대기열 + Rate limiting	순간 요청을 줄 세워 처리 속도에 맞게 흘려보냄	정상 트래픽 폭증과 DoS 공격 모두에 대한 가용성 방어
Redis 원자연산	좌석 예약을 단일 원자적 명령으로 처리	동시성(Race Condition) 취약점 구조적 제거, 무결성 보장
DB 계정 최소화	쓰기/읽기/복제 계정을 분리	계정 하나가 뚫려도 전체 데이터 장악 불가
fail2ban	반복 실패 IP 자동 차단	무차별 대입 공격 자동 방어
TLS(Origin 인증서)	구간 암호화	스니핑에 의한 정보 노출 방지, 기밀성 확보
Suricata + ELK	트래픽/로그 이상탐지 및 상관분석	공격을 사후가 아닌 진행 중에 탐지, 대응 시간 단축
서버측 금액·재고 재계산	클라이언트 값을 신뢰하지 않고 서버가 재검증	요청 변조를 통한 가격 조작·중복예약 방지

본 문서는 학습·포트폴리오 목적의 설계 문서이며, 실제 값(비밀번호, IP 대역 등)은 배포 환경에 맞게 교체해야 한다.

2.6 구현된 웹페이지 현황

페이지	기능	보안 관련성
경기 목록 / 대기열 / 좌석선택 / 결제 / 예매완료	일반 사용자 예매 플로우	Rate limiting, 동시성 제어(Redis 원자연산)
로그인 / 회원가입	회원 인증	bcrypt 해시, 인증(Authentication)
비밀번호 재설정	90일 만료 시 강제 전환	비밀번호 정책, 계정 보안 수명주기
마이페이지	본인 예매 내역 조회	접근제어(Authorization) — 주문마다 ownerEmail을 기록하고 로그인한 사용자 소유만 필터링하여 노출. 이 필터가 없으면 다른 회원의 주문이 그대로 보이는 IDOR(Insecure Direct Object Reference) 취약점이 된다.
관리자 로그인(2단계)	계정확인 → TOTP 코드 검증	다중인증(MFA)

관리자 대시보드	로그인 이력, 좌석 판매현황, 경기 등록/삭제	감사 로그(Audit Log). 관리자 전용 기능이 존재하기 때문에 MFA·IP 제한·세션 단축이 필요하다는 근거가 됨
----------	---------------------------	--

일반 회원 vs 관리자 권한 대조 — 마이페이지는 본인 주문만 필터링해서 보여주지만, 관리자 대시보드의 "총 예매 건수"는 필터 없이 전체 주문을 집계한다. 동일한 데이터(myOrders)에 대해 권한에 따라 접근 범위가 달라지는 것을 하나의 데모 안에서 직접 대조할 수 있다.

Part 3 — 위협 대응 및 계정 보안

3.1 공격 시나리오별 대응 로직

탐지·차단은 설계 위에 얹는 보조 계층이며, 핵심은 "공격 여부를 사람이 판단하기 전에 구조 자체가 최악의 상황을 안전하게 처리하도록" 만드는 것이다. 아래는 시나리오별 판단 기준과 대응 흐름이다.

3.1.1 단순 트래픽 과부하 (정상 유저가 몰리는 경우)

판단 기준 — 요청 출발지 IP가 다양하고 요청 패턴이 사람다움 → 공격이 아니라 진짜 수요.

```

사용자 몰림
|
[queue-gate] limit_req로 IP당 초당 10건 제한, burst 20까지만 허용
| (한도 초과분은 차단이 아니라 대기열로 순서 부여)
▼
[Redis 대기열] ZADD로 순번 부여 → 순번 도달 시에만 web-lb 통과
|
[web-lb] least_conn으로 was-01/02에 균등 분산

```

3.1.2 봇/매크로 공격 (좌석 스나이핑, DoS성 반복 요청)

판단 기준 — 동일 IP/세션의 비정상적으로 빠른 반복 요청, 정상 흐름(조회→선택→결제)을 건너뛰고 바로 예매 API 호출.

```

# Suricata 탐지 룰 예시
alert http any any -> any any (msg:"Excessive booking API calls";
content:"/api/book"; threshold: type threshold, track by_src,
count 20, seconds 10; sid:1000020;)

```

```

[Suricata 알림] → [ELK 로그 상관분석] → [대응]
- queue-gate에서 해당 IP 임시 차단(fail2ban 유사 룰)
- 예매 API 호출 전 첼린지(캡차/행동기반) 단계 삽입
- 세션당 예매 요청 횟수 제한 (예: 1분에 3회)

```

3.1.3 좌석 중복판매 시도 (동시성 공격/버그 악용)

판단 기준 — 같은 좌석에 여러 요청이 동시 도달하는 상황은 공격 여부와 무관하게 발생 가능 → 애초에 구조적으로 막아야 한다.

```

동시 요청 100건이 같은 좌석(A5)에 도달
|
[Redis] SETNX/DECR 원자 연산으로 "이미 잡힌 좌석"인지 단일 연산으로 판정
├─ 성공(1건) → DB에 예약 기록, 결제 진행
└─ 실패(나머지 99건) → 즉시 "이미 선택된 좌석입니다" 응답

```

3.1.4 인증 공격 (무차별 대입, 크리덴셜 스테핑)

```

[fail2ban] (bastion, queue-gate) 로그 패턴 감지 → 임계치 초과 IP 자동 ufw 차단
[was + Redis] 계정별 실패 횟수 카운트 → 5회 실패 시 30분 계정 잠금
[ELK] 로그인 실패 급증 대시보드 → 임계치 초과 시 관리자 알림

```

3.1.5 SQLi / XSS 시도

[WAS] 파라미터 바인딩(ORM/Prepared Statement) → 쿼리 조작 자체가 불가능
[WAS] 출력 이스케이프 처리 → 스크립트 삽입돼도 실행되지 않음
[Suricata/WAF] 알려진 SQLi/XSS 패턴 매칭 시 요청 차단 및 로그 기록

전체 흐름 요약 — 평시: queue-gate(속도제한) → LB(분산) → WAS(검증) → Redis(원자적 재고처리) → DB(최소권한). 이상탐지 시: Suricata/Nginx 로그 → ELK 상관분석 → 임계치 초과 IP/계정 자동 차단 → 관리자 알림.

3.2.1 설계 원칙

비밀번호 하나에 의존하는 인증은 유출·추측·재사용 공격에 취약하다. 이 문서는 회원 계정과 관리자 계정을 분리하여, 권한이 큰 관리자 계정에 더 엄격한 정책을 적용하는 것을 기본 원칙으로 한다. 다중인증(MFA)은 문자(SMS) 인증 대신 **앱 기반 TOTP(Time-based One-Time Password)** 방식을 채택했는데, 이는 외부 문자 발송 서비스 없이 서버가 시크릿 키 생성과 코드 검증만으로 동작해 **추가 비용이 들지 않고**, SIM 스와핑 등 SMS 고유의 탈취 위험도 없기 때문이다.

구분	회원 계정	관리자 계정
비밀번호 저장	bcrypt 해시	bcrypt 해시
비밀번호 만료	90일(선택 적용)	90일(필수 권장)
MFA	선택	필수(TOTP)
로그인 실패 잠금	5회/30분	5회/30분 + 알림
세션 유지시간	길게(예: 2주)	짧게(30분)
접근 IP 제한	없음	사무실/VPN IP 화이트리스트
로그인 이력 기록	선택	필수

3.2.2 회원 비밀번호 보안

3.2.2-1 DB 스키마

```
ALTER TABLE members
ADD COLUMN password_hash VARCHAR(255) NOT NULL,
ADD COLUMN password_changed_at DATETIME NOT NULL DEFAULT NOW(),
ADD COLUMN password_history JSON,
ADD COLUMN failed_login_count INT DEFAULT 0,
ADD COLUMN locked_until DATETIME NULL;
```

3.2.2-2 해시 저장 (bcrypt)

```
npm install bcrypt --save
```

```
// 회원가입
const hash = await bcrypt.hash(req.body.password, 12);
await db.query(
  'INSERT INTO members (email, password_hash, password_changed_at) VALUES (?, ?, NOW())',
  [req.body.email, hash]
);

// 로그인 검증
const match = await bcrypt.compare(req.body.password, user.password_hash);
```

3.2.2-3 비밀번호 재사용 방지

```
const isReused = await Promise.all(
  user.password_history.map(oldHash => bcrypt.compare(newPassword, oldHash))
);
```

```
);
if (isReused.includes(true)) {
  throw new Error('최근 사용한 비밀번호는 다시 사용할 수 없습니다.');
```

3.2.2-4 유출된 비밀번호 확인 (Have I Been Pwned, 무료 API)

```
# 원문 비밀번호를 전송하지 않고 SHA1 해시 앞 5자리만 조회 (k-anonymity)
echo -n "userPassword123" | sha1sum
curl https://api.pwnedpasswords.com/range/5BAA6
```

```
const crypto = require('crypto');
const sha1 = crypto.createHash('sha1').update(password).digest('hex').toUpperCase();
const [prefix, suffix] = [sha1.slice(0,5), sha1.slice(5)];
const res = await fetch(`https://api.pwnedpasswords.com/range/${prefix}`);
if ((await res.text()).includes(suffix)) throw new Error('이미 유출된 적 있는 비밀번호입니다.');
```

3.2.2-5 비밀번호 90일 만료 체크

```
// 로그인 성공 직후, 세션 발급 전
const days = Math.floor((Date.now() - new Date(user.password_changed_at)) / 86400000);
if (days > 90) {
  return res.redirect('/reset-password?forced=true');
```

3.2.3 로그인 보호 (Redis 실패 카운터)

```
redis-cli -h 10.10.0.30 -a <비밀번호> SET login_fail:user123 0 EX 1800
```

```
const key = `login_fail:${userId}`;
const count = await redis.incr(key);
await redis.expire(key, 1800);
if (count >= 5) {
  return res.status(423).json({ message: '로그인 시도 5회 초과, 30분 후 다시 시도하세요.' });
}
```

```
# 로그인 API 자체에 대한 Rate limiting (Nginx)
limit_req_zone $binary_remote_addr zone=login:10m rate=5r/m;

location /api/login {
  limit_req zone=login burst=3 nodelay;
  proxy_pass http://was_pool;
}
```

3.2.4 관리자 계정 강화

3.2.4-1 관리자 테이블 및 로그인 이력

```
CREATE TABLE admins (
  id INT AUTO_INCREMENT PRIMARY KEY,
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  mfa_secret VARCHAR(255) NOT NULL,
  password_changed_at DATETIME NOT NULL DEFAULT NOW(),
  created_at DATETIME DEFAULT NOW()
);

CREATE TABLE admin_login_log (
  id INT AUTO_INCREMENT PRIMARY KEY,
  admin_id INT NOT NULL,
  ip_address VARCHAR(45),
```

```
success BOOLEAN,  
logged_at DATETIME DEFAULT NOW(),  
FOREIGN KEY (admin_id) REFERENCES admins(id)  
);
```

3.2.4-2 MFA — 앱 기반 TOTP (무료, SMS 미사용)

```
npm install speakeasy qrcode
```

```
// 최초 등록 — Google Authenticator/Authy 같은 앱에 QR 등록  
const speakeasy = require('speakeasy');  
const qrcode = require('qrcode');  
  
const secret = speakeasy.generateSecret({ name: 'TicketAdmin' });  
await db.query('UPDATE admins SET mfa_secret=? WHERE id=?', [secret.base32, adminId]);  
const qrlImage = await qrcode.toDataURL(secret.otpauth_url);  
  
// 로그인 시 6자리 코드 검증  
const verified = speakeasy.totp.verify({  
  secret: admin.mfa_secret,  
  encoding: 'base32',  
  token: req.body.code,  
  window: 1,  
});
```

SMS 대신 TOTP를 쓰는 이유 — TOTP는 외부 문자 발송 서비스(Twilio 등) 연동이 필요 없어 건당 비용이 전혀 발생하지 않고, 서버가 시크릿 키와 시간 기반 알고리즘만으로 코드를 검증하므로 SIM 스와핑 같은 SMS 고유의 탈취 경로도 원천적으로 차단된다.

3.2.4-3 관리자 세션 타임아웃 (짧게 설정)

```
npm install express-session connect-redis
```

```
const session = require('express-session');  
const RedisStore = require('connect-redis').default;  
  
app.use('/admin', session({  
  store: new RedisStore({ client: redisClient }),  
  secret: process.env.SESSION_SECRET,  
  cookie: { maxAge: 30 * 60 * 1000, secure: true, httpOnly: true },  
}));
```

3.2.4-4 관리자 페이지 접근 IP 제한 (Nginx)

```
# /etc/nginx/conf.d/admin.conf  
location /admin {  
  allow <사무실_또는_VPN_IP>;  
  deny all;  
  proxy_pass http://was_pool;  
}
```

```
sudo nginx -t && sudo systemctl reload nginx
```

3.2.5 시크릿 관리

```
cat > .env << 'EOF'  
DB_PASSWORD=$(openssl rand -base64 24)  
SESSION_SECRET=$(openssl rand -hex 32)  
REDIS_PASSWORD=$(openssl rand -base64 24)  
EOF  
chmod 600 .env  
echo ".env" >> .gitignore
```

3.2.6 정책 요약

설정	효과
bcrypt 해시	DB 유출 시에도 비밀번호 원문 노출 방지
비밀번호 재사용 금지	만료 정책을 예측 가능한 패턴으로 우회하는 것 방지
HIBP 유출 확인	이미 공개된 비밀번호 사용을 원천 차단
Redis 로그인 잠금	무차별 대입 공격 자동 방어
관리자 TOTP(MFA)	비밀번호 유출만으로는 관리자 계정 탈취 불가, 비용 없음
관리자 IP 화이트리스트	지정된 위치 외에서는 관리자 페이지 자체에 접근 불가
관리자 로그인 이력	이상 접속 탐지 및 사후 추적 가능

본 문서는 학습·포트폴리오 목적의 설계 문서이며, 시크릿 값은 예시이므로 실제 배포 시 반드시 교체해야 한다.

Part 4 — 검증 계획

4.1 단계별 실행 계획 (Phase 1~6)

Phase 1 — 인프라 최소 구축

동시성 검증(Phase 3)만 목표라면 아래 4대만 있으면 충분하다. LB·대기열은 Phase 4에서 추가한다.

VM	IP	역할
bastion	10.10.0.10	SSH 유일 진입점
was-01	10.10.0.11	예약 API 서버
redis	10.10.0.30	좌석 재고 원자연산
db-01	10.10.0.40	예약 기록 저장(MariaDB)

- 새 VMnet(예: VMnet10, 10.10.0.0/24) 생성, 호렘 네트워크와 분리
- 각 VM Ubuntu 22.04 설치 후 Netplan으로 고정 IP 설정
- ufw로 각 서버 화이트리스트 적용 (bastion 22번만 허용 원칙)
- Redis requirepass 설정, bind-address 내부 IP로 제한
- MariaDB 최소권한 계정(was_writer) 생성

Phase 2 — 예약 API + Redis 원자연산

좌석 예약의 핵심은 "확인 후 예약"이 아니라 "예약 시도 자체가 원자적 연산"이어야 한다는 점이다.

```
# 의사코드 — 좌석 예약 원자적 처리
POST /api/book-seat { gameId, seatId }

result = redis.SETNX("seat:{gameId}:{seatId}", userId)
IF result == 1:
    DB.INSERT booking(gameId, seatId, userId)
    RETURN 200 "예약 성공"
```

```
ELSE:
  RETURN 409 "이미 선택된 좌석입니다"
```

Phase 3 — 시나리오 3: 동시성 검증 (최우선 진행)

같은 좌석(A5)에 100개의 요청을 정확히 동시에 보내, 성공 응답이 1건만 발생하는지 확인한다.

```
// seat-race.js
import http from 'k6/http';
import { check } from 'k6';

export const options = {
  scenarios: {
    same_seat_attack: {
      executor: 'shared-iterations',
      vus: 100,
      iterations: 100,
      maxDuration: '10s',
    },
  },
};

export default function () {
  const payload = JSON.stringify({ gameId: 1, seatId: 'A5' });
  const res = http.post('http://10.10.0.11:3000/api/book-seat', payload, {
    headers: { 'Content-Type': 'application/json' },
  });
  check(res, { '성공 또는 이미선택 응답': (r) => [200, 409].includes(r.status) });
}
```

판정 기준 — 테스트 후 `SELECT seat_id, COUNT(*) FROM bookings WHERE seat_id='A5'` 결과가 **1이면 정상**, 2 이상이면 원자연산 로직에 결함이 있는 것이므로 Redis 로직부터 재점검한다.

Phase 4 — 인프라 확장

Phase 3 검증이 끝난 뒤, 트래픽 분산·제한 계층을 추가한다.

추가 VM	IP	역할
queue-gate	10.10.0.15	Rate limiting, 대기열(Nginx + Redis ZADD)
web-lb	10.10.0.20	Nginx 로드밸런서, was-01/02 분산
was-02	10.10.0.12	예약 API 서버(수평 확장)

Phase 5 — 시나리오 2: 폭증 트래픽 비교 (대기열 ON/OFF)

```
// booking-spike.js
export const options = {
  stages: [
    { duration: '10s', target: 100 },
    { duration: '5s', target: 2000 },
    { duration: '1m', target: 2000 },
    { duration: '20s', target: 0 },
  ],
};

export default function () {
  const res = http.get('http://10.10.0.15/booking'); // queue-gate 경유
  check(res, { '정상 응답': (r) => r.status === 200 || r.status === 429 });
}
```

① **대기열 OFF** (queue-gate의 limit_req 비활성화 후 실행)

```
k6 run --out json=no-queue.json booking-spike.js
```

② **대기열 ON** (limit_req 활성화 후 동일 시나리오 실행)

```
k6 run --out json=with-queue.json booking-spike.js
```

두 결과의 **p95 응답시간**과 **에러율(http_req_failed)**을 비교하여, 대기열 도입 효과를 수치와 그래프로 제시한다.

Phase 6 — 문서화 및 포트폴리오 정리

- 프로젝트 개요서 (필요성 / CIA 3요소 연계 / 얻은 점)
- 아키텍처 다이어그램 (기본 구성 → 트래픽 대응 확장 구성)
- 위협모델링 문서 (STRIDE 표)
- 공격 대응 로직 문서 (과부하 / 봇 / 동시성 / 인증공격별 대응)
- 부하테스트 결과 보고서 (시나리오 3, 2 결과 + 그래프)
- 트러블슈팅 노트 (문제-원인-해결 2~3건)
- GitHub README 및 리포지토리 구조 정리

4.2 성공 판단 기준

항목	기준	실패 시 점검 포인트
좌석 중복판매	동시 100건 요청 시 성공 응답 정확히 1건	Redis 원자연산 로직, 트랜잭션 경계
대기열 효과	대기열 ON일 때 에러율이 OFF 대비 유의미하게 낮음	limit_req 설정값, Redis 큐 처리 로직
응답시간(p95)	폭증 구간에서도 대기 응답(429/큐잉)이 타임아웃보다 우선 발생	WAS 커넥션 풀, Nginx 타임아웃 설정

본 문서는 학습·포트폴리오 목적의 진행 계획서이며, 실제 수치는 구축 환경(VM 사양, 네트워크 대역폭)에 따라 달라질 수 있다.

Part 5 — 자체 모의해킹 검증

5.1 목적 및 범위

지금까지는 "이렇게 설계하면 안전하다"는 방어자 관점으로만 접근했다. 이 단계에서는 반대로 **공격자 관점에서 직접 정찰하고, 스캔하고, 취약점을 시도해보며** 설계한 방어가 실제로 작동하는지 검증한다. 이 과정과 결과를 문서화하면 "설계했다"가 아니라 "검증까지 했다"는 근거가 된다.

범위 및 준수사항

- ① 대상은 본인이 구축한 VMnet10(10.10.0.0/24) 내부 VM으로 한정한다.
- ② 인터넷에 노출된 실제 서비스, 제3자 소유 시스템, 프로덕션 환경에는 절대 사용하지 않는다.
- ③ 정보통신망법상 정당한 접근권한 없는 시스템에 대한 침입·스캔은 처벌 대상이며, 본 방법론은 **자기 소유 인프라에 대한 학습 목적 검증**에 한해서만 유효하다.

5.2 방법론 개요 (Cyber Kill Chain 축약)

단계	목표	사용 도구
A. 정찰(Recon)	열린 포트, 실행 서비스, 버전 파악	nmap
B. 웹 취약점 스캔	알려진 웹 취약점 자동 탐지	Nikto, OWASP ZAP
C. 인증 공격 시뮬레이션	무차별 대입 방어(fail2ban, 계정 잠금) 검증	Hydra
D. 입력값 검증 취약점	SQLi/XSS 방어(Prepared Statement) 검증	sqlmap, 수동 페이로드
E. 동시성 취약점 재검증	이전 K6 시나리오 3 결과와 교차 확인	K6(기 구축 스크립트)
F. 보고서 작성	발견사항 정리 및 심각도 평가	본 문서 5.6 템플릿

5.3 도구 설치

```
sudo apt update
sudo apt install -y nmap nikto hydra sqlmap

# OWASP ZAP은 Docker로 실행 (설치가 간단하고 버전 관리 용이)
sudo apt install -y docker.io
sudo docker pull zaproxy/zap-stable
```

5.4 단계별 실행 방법

4-A. 정찰 (nmap)

```
# 어떤 포트가 실제로 열려있는지 확인 (설계와 실제가 일치하는지 대조)
nmap -sV -sC -p- 10.10.0.20 -oN recon-weblb.txt
nmap -sV -sC -p- 10.10.0.11 -oN recon-was01.txt

# 알려진 취약점 스크립트로 추가 확인
nmap --script vuln 10.10.0.20
```

판단 기준 — 설계상 web-lb는 80/443만 열려 있어야 한다. nmap 결과에 3000, 6379, 3306 같은 내부 포트가 보이면 방화벽 설정에 결함이 있는 것이다.

4-B. 웹 취약점 스캔 (Nikto, OWASP ZAP)

```
nikto -h http://10.10.0.20 -output nikto-report.txt

# ZAP 자동 베이스라인 스캔 (Docker)
sudo docker run -t zaproxy/zap-stable zap-baseline.py \
-t http://10.10.0.20 -r zap-report.html
```

ZAP 리포트는 서버 응답 헤더 누락(HSTS, X-Frame-Options 등), 쿠키 보안 속성 누락 등을 자동으로 짚어준다. 이전에 설정한 Nginx 보안 헤더가 실제로 응답에 포함되는지 여기서 확인 가능하다.

4-C. 인증 공격 시뮬레이션 (Hydra)

```
# 로그인 API에 대해 무차별 대입 시도 (본인 서버 대상)
hydra -l demo@test.com -P /usr/share/wordlists/rockyou.txt \
10.10.0.20 http-post-form \
"/api/login:email=^USER^&password=^PASS^:F=Invalid"
```

판단 기준 — 5회 시도 이후 423 Locked 응답이 오고, fail2ban/ufw 로그에 해당 IP 차단 기록이 남아야 정상이다. Hydra가 수백 번 시도했는데도 계속 200으로 응답한다면 로그인 잠금 로직에 결함이 있는 것이다.

```
# fail2ban 차단 여부 확인
sudo fail2ban-client status sshd
sudo ufw status | grep <공격_시뮬레이션_IP>
```

4-D. SQLi / XSS 검증

```
# 로그인 파라미터에 SQLi 자동 탐지 시도
sqlmap -u "http://10.10.0.20/api/login" \
--data="email=test@test.com&password=test" \
--risk=2 --level=3 --batch

# 작성 검색 API 등 GET 파라미터가 있다면
sqlmap -u "http://10.10.0.20/api/games?id=1" --batch
```



```
# XSS는 수동으로 입력 필드에 페이로드 삽입 후 반영 여부 확인
<script>alert(1)</script>
">
```

판단 기준 — sqlmap이 "injectable parameter"를 찾지 못해야 정상(Prepared Statement가 제대로 적용된 것). XSS 페이로드가 화면에 그대로 실행되지 않고 문자로만 표시되면 이스케이프 처리가 정상 동작하는 것이다.

4-E. 동시성 취약점 재검증

새로 발견한 이슈가 없는지, 기존 K6 시나리오 3(좌석 동시클릭 100건) 스크립트를 다시 실행해 교차 검증한다.

```
k6 run seat-race.js
# 이후 DB 재확인
SELECT seat_id, COUNT(*) FROM bookings WHERE seat_id='A5' GROUP BY seat_id;
```

5.5 스캔 결과 판단 기준 요약

점검 항목	정상(방어 성공)	비정상(취약점 발견)
포트 스캔	80/443(web-lb), 22(bastion만) 외 응답 없음	3000/6379/3306 등 내부 포트가 응답
보안 헤더	HSTS, X-Frame-Options, CSP 모두 존재	헤더 누락
무차별 대입	5회 실패 후 423 잠금 + IP 차단	제한 없이 계속 시도 가능
SQLi	sqlmap이 취약 파라미터 못 찾음	injectable parameter 발견
XSS	입력값이 문자로만 표시됨	스크립트가 실제 실행됨
좌석 동시성	중복판매 0건	동일 좌석 2건 이상 예약

5.6 취약점 보고서 템플릿

발견된 사항은 아래 형식으로 기록한다. 실제로 취약점이 없더라도 "점검했고 이상 없음을 확인했다"는 기록 자체가 포트폴리오 근거 자료가 된다.

ID	항목	심각도	재현 방법	영향 / 조치
VULN-01	예시: 보안 헤더 누락	Medium	ZAP 베이스라인 스캔	클릭재킹 위험 → X-Frame-Options 추가 조치
VULN-02	예시: (해당사항 없음)	Low	-	-

- 발견된 취약점은 CVSS 간이 점수(Critical/High/Medium/Low)로 심각도 표기
- 재현 절차를 스크린샷과 함께 기록 (nmap 결과, ZAP 리포트, sqlmap 출력 등)
- 조치 전/후 스캔 결과를 나란히 비교 (Before/After)
- 발견 없음도 "점검 완료" 항목으로 명시 (아무것도 안 한 것과 다름을 보여줌)

5.7 윤리 및 법적 유의사항

정보통신망 이용촉진 및 정보보호 등에 관한 법률(정보통신망법) 제48조는 정당한 접근권 없이 정보통신망에 침입하는 행위를 금지한다. 본 방법론에서 사용하는 nmap, sqlmap, hydra 등은 **본인이 소유하고 격리된 VMnet 내부에서, 본인이 만든 서비스에 한해서만** 사용해야 하며, 인터넷상의 제3자 서비스나 권한 없는 시스템에는 절대 사용해서는 안 된다.

본 문서는 학습-포트폴리오 목적의 방법론 문서이며, 실제 수행은 반드시 격리된 자체 인프라 내에서만 진행한다.

Part 6 — 산출물

6.1 문서 산출물

- 프로젝트 개요서 (필요성, CIA 3요소 연계, 얻은 점)
- 위협모델링 및 공격 대응 로직 문서 (본 문서 3장)
- 서버 구성도 및 인프라 설계 문서 (본 문서 2장)
- 방화벽/포트 설정 정리 문서 (본 문서 2.4)
- 회원·관리자 인증 보안 설정 문서 (본 문서 3.2)
- 부하테스트 결과 보고서 (시나리오 3: 동시성 검증, 시나리오 2: 대기열 유무 비교)
- 트러블슈팅 노트 (문제-원인-해결 정리)
- 자체 모의해킹 취약점 진단 보고서 (nmap/ZAP/sqlmap/hydra 결과 및 조치내역)

6.2 구현 산출물

- 야구 티켓 예매 웹페이지 (프론트엔드, 좌석선택~결제 플로우)
- 좌석 예약 API (Redis 원자연산 기반 동시성 제어 로직)
- VM 인프라 구성 (Bastion, WAS, Redis, DB, queue-gate, LB, 모니터링)
- 각 서버 설정 스크립트 (ufw, nginx conf, mariadb 계정 생성 SQL 등)
- K6 부하테스트 스크립트 (동시성 테스트, 트래픽 폭증 테스트)
- 회원가입/로그인 인증 로직 코드 (bcrypt, 로그인 잠금, 관리자 TOTP MFA)
- 모의해킹 스캔 스크립트 및 원본 결과 로그 (nmap/nikto/zap-report.html 등)

6.3 시각 자료 및 발표용

- 서버 구성도 다이어그램
- 부하테스트 결과 그래프 (대기열 있음/없음 비교)
- 동시성 검증 결과 캡처 (DB 조회 결과)
- ufw 방화벽 상태 및 로그인 이력 대시보드 스크린샷
- GitHub 리포지토리 (코드 + 문서 + README)
- PPT 슬라이드 (개요 → 아키텍처 → 위협모델 → 검증결과 요약)
- 시연 영상 또는 GIF (예매 플로우 + 대기열 동작)

본 문서는 학습·포트폴리오 목적으로 지금까지 논의한 설계·구현·검증 내용을 하나로 정리한 통합본이며, 실제 값(비밀번호, IP 대역 등)은 배포 환경에 맞게 교체해야 한다.

Part 7 — 캡처 자료 체크리스트

포트폴리오 근거 자료로 남겨야 할 작업 결과 캡처 목록이다. 4번(모의해킹)과 5번(동시성 검증)을 가장 먼저, 가장 꼼꼼히 남기는 것을 우선순위로 한다.

7.1 인프라 구축

- ☐ VMware 가상 네트워크 편집기 — 새 VMnet 생성 화면
- ☐ 각 VM ip a 실행 결과 (고정 IP 적용 확인)
- ☐ netplan apply 실행 화면
- ☐ 각 서버 sudo ufw status verbose 결과
- ☐ SSH 키 등록 후 ssh was-01 한 줄로 bastion 경유 접속되는 터미널 화면
- ☐ SHOW GRANTS FOR 'was_writer'@'10.10.0.%' (계정 권한 분리 확인)
- ☐ SHOW SLAVE STATUS\G (DB 복제 정상 동작 확인)
- ☐ Redis redis-cli -a <비밀번호> PING (인증 적용 확인)

7.2 웹페이지 배포

- `curl -I http://10.10.0.20` 응답 헤더 (보안 헤더 포함 여부)
- 예매 플로우 전체 스크린샷 (경기목록 → 대기열 → 좌석선택 → 결제 → 완료)
- 로그인/회원가입 화면
- 관리자 로그인 2단계(TOTP 입력) 화면
- 관리자 대시보드 (판매현황, 로그인이력, 경기관리)

7.3 인증/계정 보안 검증

- 로그인 전 헤더(로그인 버튼) → 로그인 성공 후 헤더(유저칩) Before/After 비교
- 개발자도구 Network 탭 — 로그인 API 요청/응답
- 개발자도구 Application 탭 — 쿠키 속성(HttpOnly/Secure/SameSite) 확인
- bcrypt 해시가 저장된 DB 테이블 조회 결과 (평문이 아님을 증명)
- `Redis login_fail:*` 키 조회 (실패 카운터 동작 확인)
- 5회 실패 후 423 Locked 응답 캡처
- `fail2ban-client status sshd` (차단된 IP 목록)
- 계정A/계정B로 각각 로그인 후 마이페이지에 서로 다른 주문만 보이는 화면 (접근제어 증거)

7.4 자체 모의해킹 (최우선)

- `nmap -sV -sC -p- 10.10.0.20` 스캔 결과 (의도한 포트만 열려있는지)
- Nikto 스캔 결과 (nikto-report.txt)
- OWASP ZAP 리포트 (zap-report.html), 특히 보안 헤더 항목
- Hydra 무차별 대입 시도 로그 (몇 번째 시도부터 차단됐는지)
- sqlmap 실행 결과 ("injectable parameter not found" 문구)
- 취약점 조치 전/후 재스캔 결과 비교 (Before/After)

7.5 동시성 검증 (K6 시나리오 3, 최우선)

- `k6 run seat-race.js` 실행 중 터미널 출력 (VU 100, iterations 100)
- 실행 후 `SELECT seat_id, COUNT(*) FROM bookings WHERE seat_id='A5'` 결과가 1인 것 확인
- (대조군) Redis 로직을 뺀 상태에서 중복판매가 실제로 발생하는 결과

7.6 트래픽 폭증 비교 (K6 시나리오 2)

- 대기열 OFF 상태 부하테스트 결과 (에러율/응답시간)
- 대기열 ON 상태 부하테스트 결과
- 두 결과를 나란히 비교한 그래프

7.7 모니터링/탐지

- Suricata 알람 로그 (/var/log/suricata/fast.log 등)
- Kibana 대시보드 (로그인 실패 추이, 트래픽 패턴 그래프)
- ELK에 여러 서버 로그가 집계되는 화면

본 문서는 지금까지 논의한 설계·구현·검증·캡처 계획 전체를 하나로 정리한 최종 통합본이다. 실제 값(비밀번호, IP 대역 등)은 배포 환경에 맞게 교체해야 한다.